

Introduction:

This project evaluates and compares multiple ML pipelines for text classification using the Nvidia Nemotron Model Reasoning Challenge dataset from kaggle. The goal is to analyze how different text representation methods and machine learning algorithms perform in terms of predictive accuracy and computational efficiency. We compare traditional machine learning models using TF-IDF features as well as a neural network model using Word2Vec embeddings.

Data Preprocessing steps:

The dataset consists of structured reasoning and instruction text prompts annotated with a task_type label. The classification task is to predict the correct reasoning category from the text. During the preprocessing, I did these things:

- Conversion of all text to lowercase
- Remove task-related words (summarize, translate, etc..)
- Remove of punctuation and special characters
- Removal of extra whitespace

Evaluation Metrics Analysis:

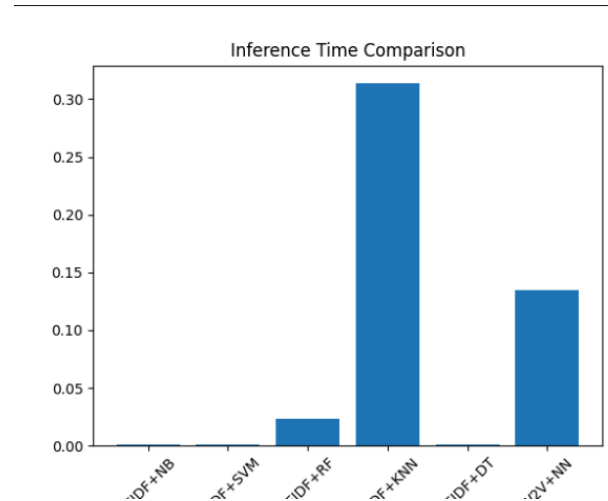
The models were evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Training time
- testing/inference time

Results:

```
Results:
TFIDF+NB: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=0.013s, TestTime=0.001s
TFIDF+SVM: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=0.788s, TestTime=0.001s
TFIDF+RF: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=0.277s, TestTime=0.023s
TFIDF+KNN: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=0.007s, TestTime=0.314s
TFIDF+DT: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=0.063s, TestTime=0.001s
W2V+NN: Acc=1.000, Prec=1.000, Rec=1.000, F1=1.000, TrainTime=3.168s, TestTime=0.135s
```

Each model has the same accuracy, precision, recall, and f-1 score of 1.000. This shows that the dataset is highly separable using simple lexical patterns. TF-IDF features alone are sufficient to distinguish between task types due to strong keyword signals in the text. Because of this, more complex models such as Random Forest and Neural Networks do not improve classification performance.



Although the accuracy is identical across models, computational efficiency is different:

- TF-IDF + Naives Bayes = fastest training and inference
- TF-IDF + SVM = slowest training among TF-IDF models
- TF-IDF+ Random Forest = moderate training time, slightly slower inference
- TF-IDF + KNN = very fast training but slow inference
- TF-IDF + Decision Tree = balance but slightly slower than Naives Bayes
- Word2Vec + Neural Network = slowest due to embedding computation and iterative training

Training Time Comparison: NB<DT<RF<SVM<NN

This shows a clear trade-off between model complexity and computational cost, even though predictive accuracy remains constant.

Comparison figures

The following visualizations were generated and included in the /report folder:

- accuracy.png = shows identical accuracy across all models
- speed.png = compares training time differences
- test_speed.png = compares inference time differences
- tradeoff.png = visualizes accuracy vs computational cost

Interpretation of Figures

- Accuracy plot shows no variation across models
- Speed plots show clear differences in computational cost
- Tradeoff plot highlights that increased complexity does not improve accuracy

These figures confirm that the primary difference between models is efficiency rather than predictive capability.

Conclusion

All models achieved perfect classification accuracy, which indicates that the dataset is highly structured and easily separable using surface-level lexical patterns. In this case, advanced models such as neural networks do not provide additional predictive benefit.

Recommended Model

TF-IDF + Naive Bayes

Justification:

- Achieves perfect accuracy (1.0)
- Fastest training time among all models
- Extremely fast inference time
- Simple and computationally efficient
- No performance gain from more complex models

Final Conclusion

This experiment demonstrates an important machine learning principle: When a dataset is highly separable using simple features, increasing model complexity does not improve accuracy, it only increases computational cost. Therefore, for this specific task, simpler models such as Naive Bayes combined with TF-IDF are the most practical and efficient solution.